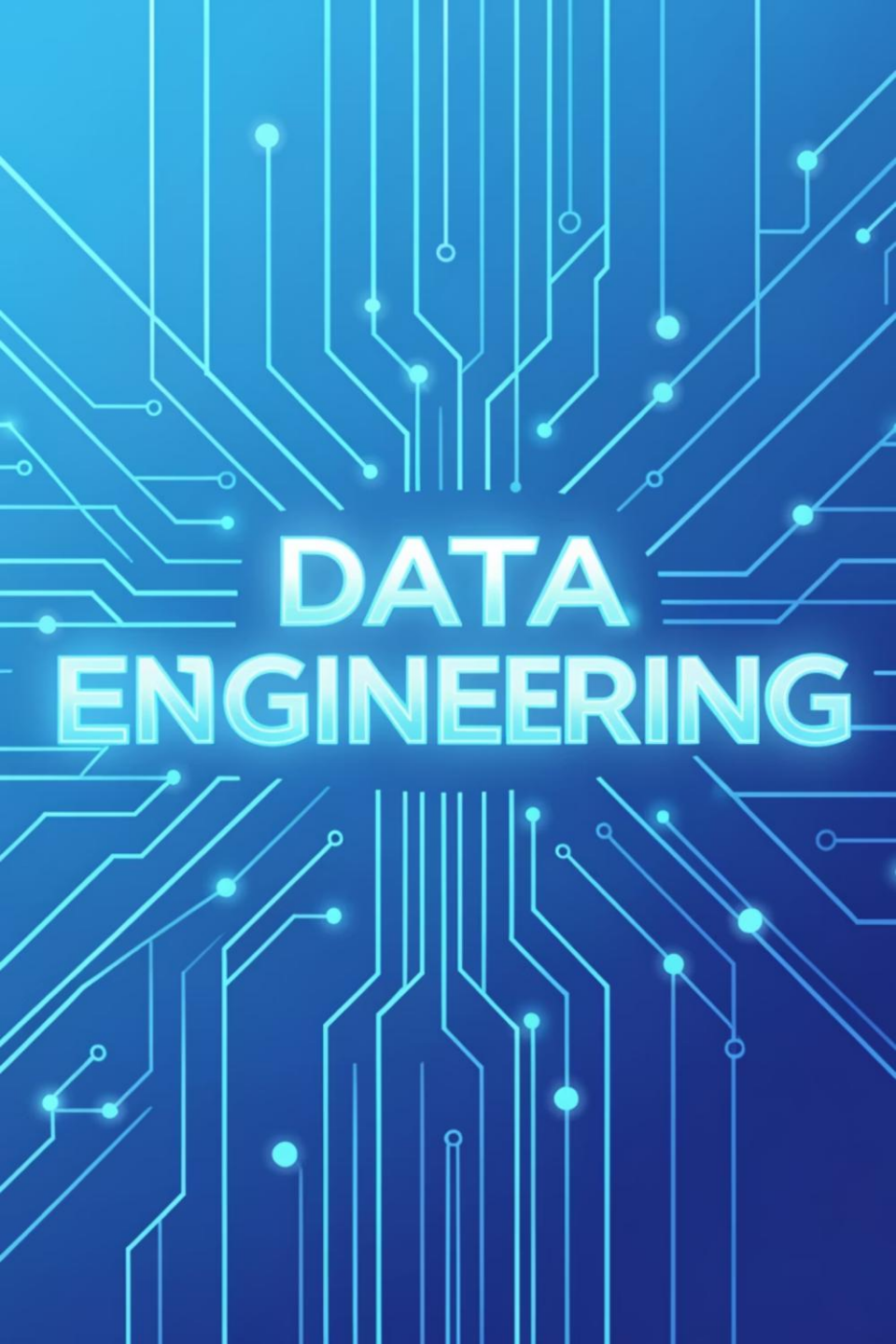




DATA ENGINEERING

Data Engineering Spring '26 – DS463

Feb 17, 2026

Lecture 14

Logistics: Your Course Essentials

Class Timings

Monday (LH6), Tuesday (LH8): 11:30 AM
– 12:20 PM

Friday (LH8): 11:00 AM – 11:50 AM

Office Hours

Monday, Tuesday: 10:00 AM – 11:00 AM,
AM, S18 2nd Floor NAB or by
appointment.

Credit Hours

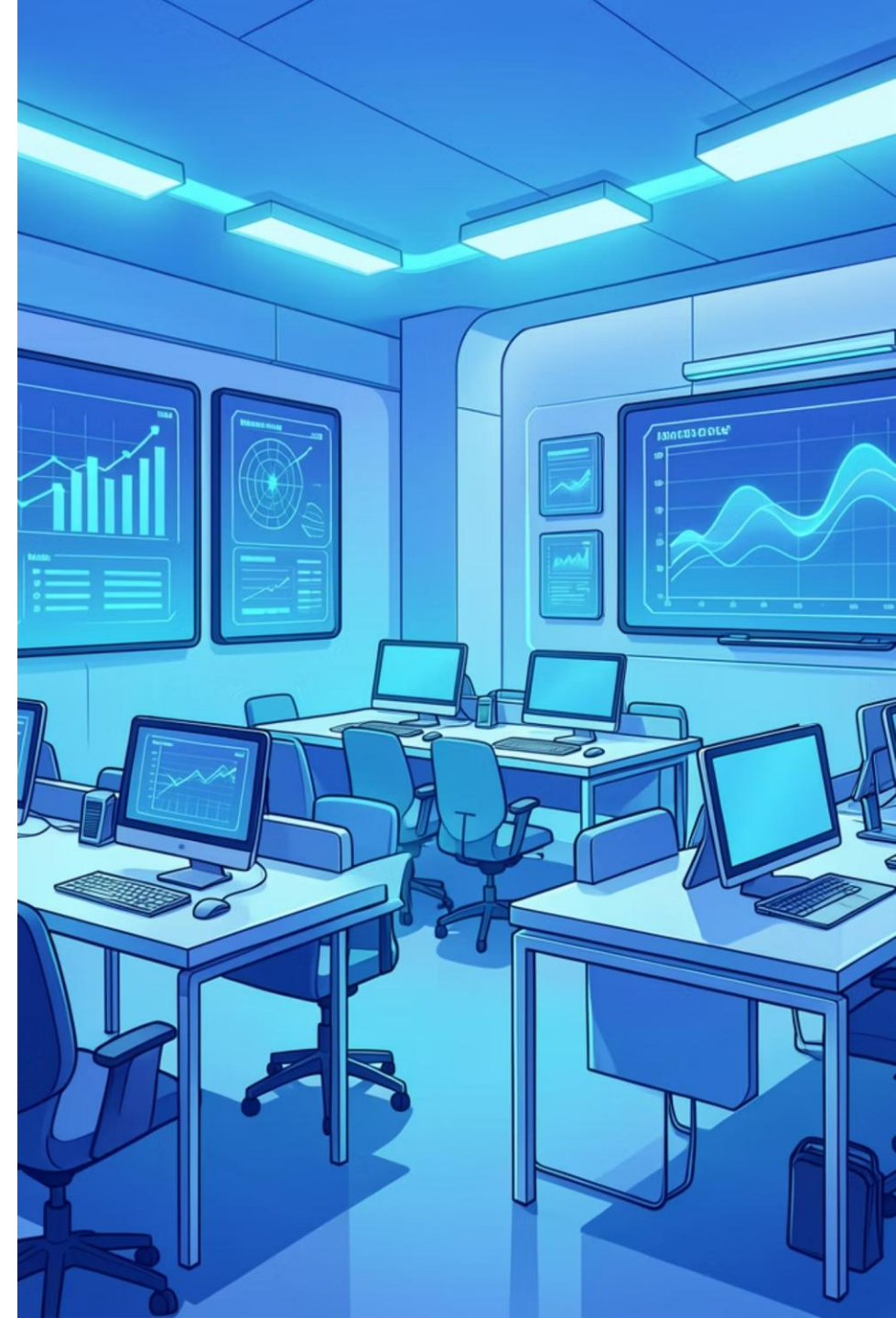
2 hours (Lectures) + 1 hour (Practical
(Practical Session)

Contact & Support

MS Teams: **w5t93yk**

Email: farah.saeed@giki.edu.pk

WhatsApp: Reach out for quick queries



Principles of Good Data Architecture

- Choose common components wisely
- Architecture is leadership

**How data architecture impacts
other teams and individuals**

- Always be architecting
- Build loosely coupled systems
- Make reversible decisions

Data architecture is ongoing process

- Plan for failure
- Architect for scalability
- Prioritize security
- Embrace FinOps

**Unspoken but
understood
priorities**

Major Architecture Concepts

Domains and Services

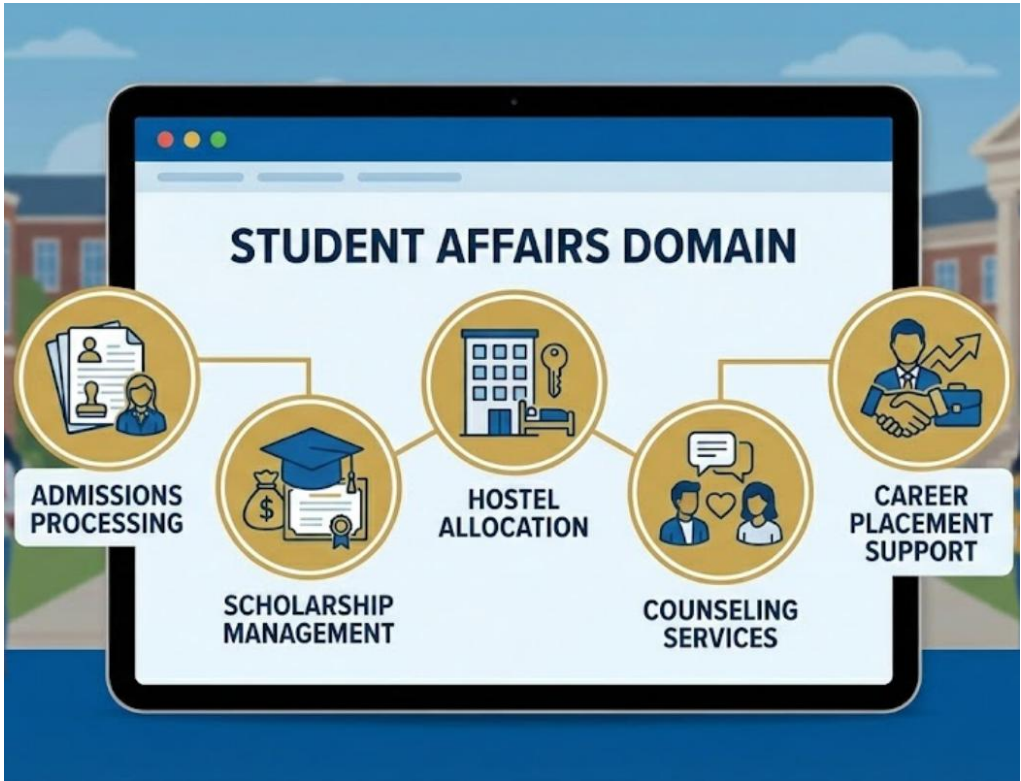
Domain

- A **domain** is the real-world subject area your software is built for.
- Broad functional area of responsibility
- To define a domain correctly:
 - Talk to users and stakeholders.
 - Understand real business processes.
 - Avoid designing in isolation (“architecting in a vacuum”).

Service

- A service performs a specific task or function.
- It has one clear responsibility.
- Services are building blocks inside a domain.

Domains and Services



Domains and Services

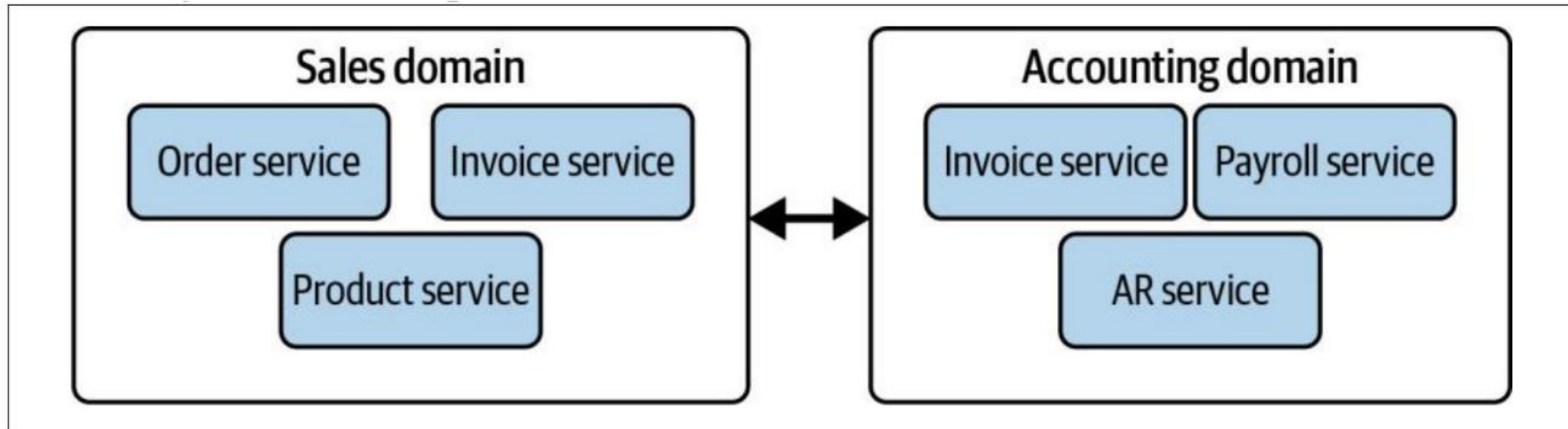


Figure 3-3. Two domains (sales and accounting) share a common service (invoices), and sales and accounting own their respective domains

Sometimes, services can be shared across domains.
Each domain still owns its business responsibilities, even if services overlap.

Distributed Systems, Scalability, and Designing for Failure

Characteristics of Data Systems

Scalability

- Ability to **increase system capacity**.
- Handles more:
 - Users
 - Queries
 - Data
- Example: Add resources to process large datasets.

Elasticity

- Ability to **automatically scale up or down** based on workload.
- Scale up → handle high demand.
- Scale down → save money (especially in cloud).
- Some systems can **scale to zero** when idle.

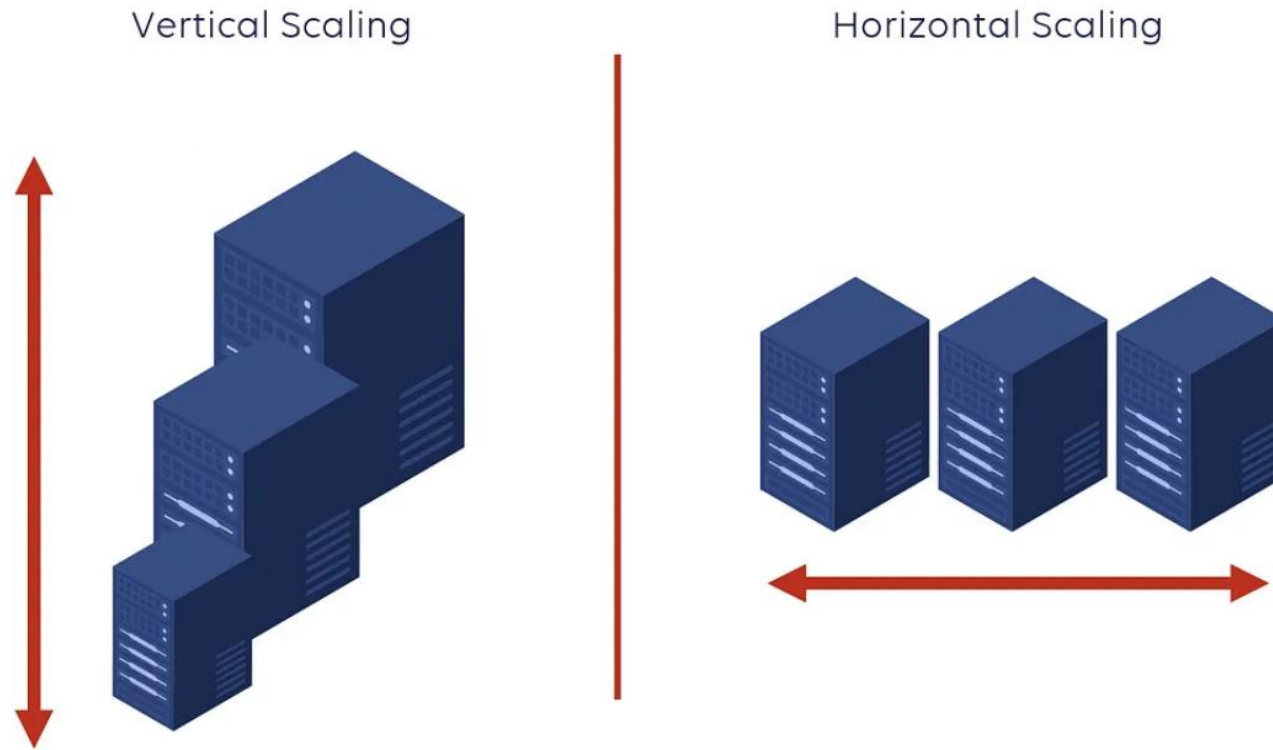
Availability

- Percentage of time the system is **up and operational**.
- High availability = minimal downtime.

Reliability

- Probability the system **performs correctly over time**.
- Low reliability → system may crash → reduces availability.
- Elasticity improves reliability by preventing overload.

Distributed Systems, Scalability, and Designing for Failure



Vertical Scaling (Single Machine)

- Add more CPU, memory, disk to one machine.
- Has physical limits.
- If the machine fails → system goes down.
- Low fault tolerance.

Horizontal Scaling (Distributed Systems)

- Add more machines.
- Work is distributed across nodes.
- Leader node assigns tasks to worker nodes.
- If one machine fails → others continue.
- Data is replicated for redundancy

Distributed Systems, Scalability, and Designing for Failure

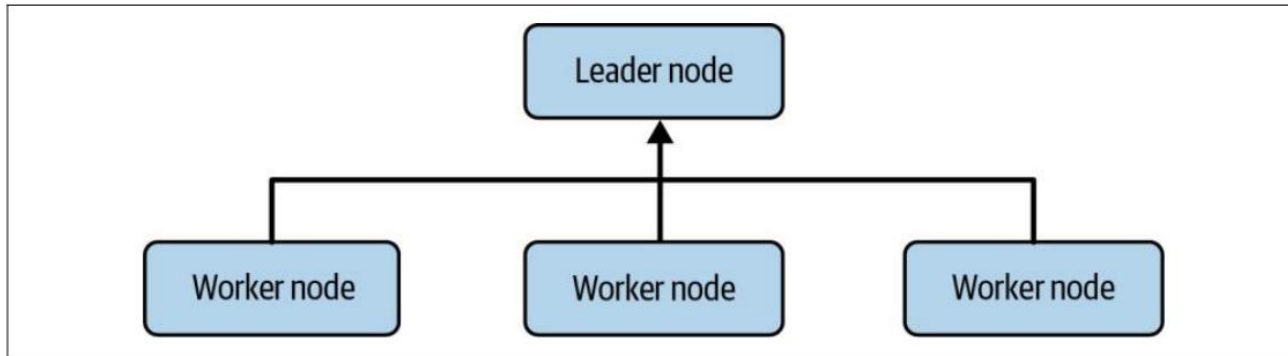


Figure 3-4. A simple horizontal distributed system utilizing a leader-follower architecture, with one leader node and three worker nodes

Why Distributed Systems Matter

- Increase:
 - Scalability
 - Availability
 - Reliability
- Provide backup and failover.
- Used in:
 - Cloud data warehouses
 - Object storage systems
- Often hidden from users, but understanding them improves performance tuning.

Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Tight coupling

- Every part of a domain and service is vitally dependent upon every other domain and service.
- Changes in one part affect many others.

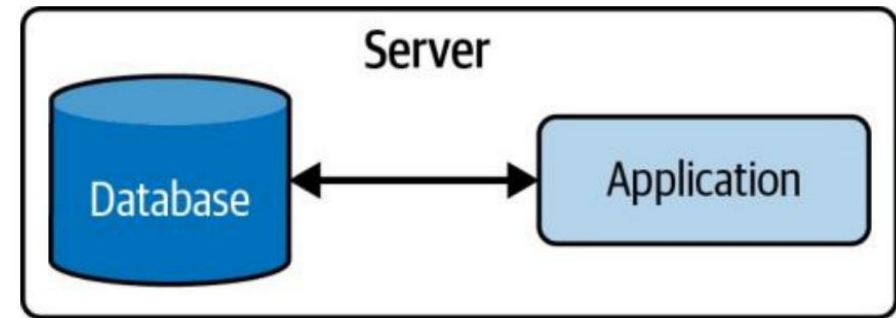
Loose coupling

- Decentralized domains and services that do not have strict dependence on each other.
- Each part can evolve or fail without breaking others.
- Requires clear standards, ownership, and accountability.
- Trade-off: too loose can cause inconsistency or poor data sharing.

Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Single Tier Architecture

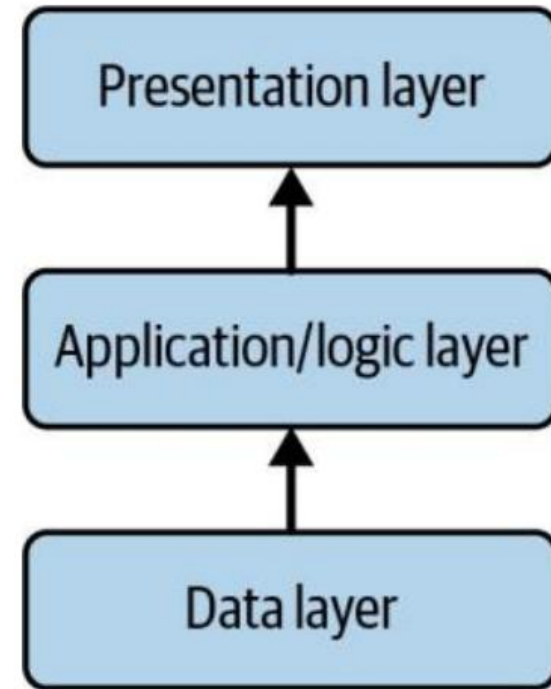
- Database and application run on one machine.
- Everything is tightly coupled.
- Simple and good for:
 - Prototyping
 - Development
- Not good for production because:
 - If one component fails → entire system fails
 - Limited scalability
 - Resource contention (CPU, memory, disk)
 - Analytics queries can overload production systems



Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Multitier (n Tier) Architecture

- Separates layers (data, logic, presentation).
- Common example: Three-tier architecture
 - Data tier
 - Application logic tier
 - Presentation tier
- Benefits:
 - Separation of concerns
 - Independent scaling
 - Better reliability
 - More flexibility in choosing technologies
- Start simple, but evolve to multitier as complexity grows.



Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Distributed Systems Considerations when scaling horizontally

Shared-Nothing Architecture

Each node has its own memory, disk, CPU.

No resource sharing.

Avoids resource contention.

Shared Disk Architecture

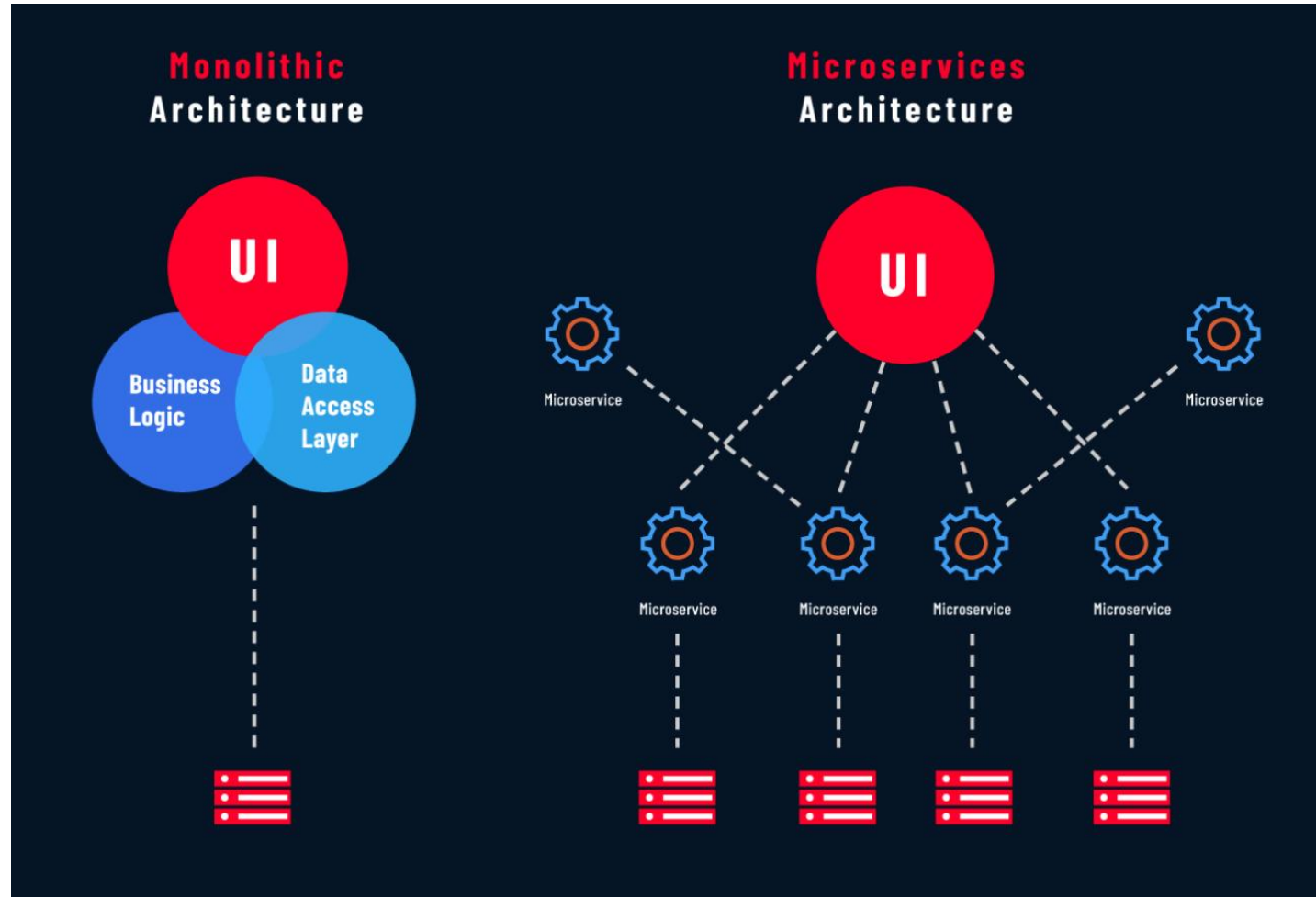
Nodes share storage.

Helpful for failover and redundancy.

Risk of resource contention.

Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Monoliths and Microservices

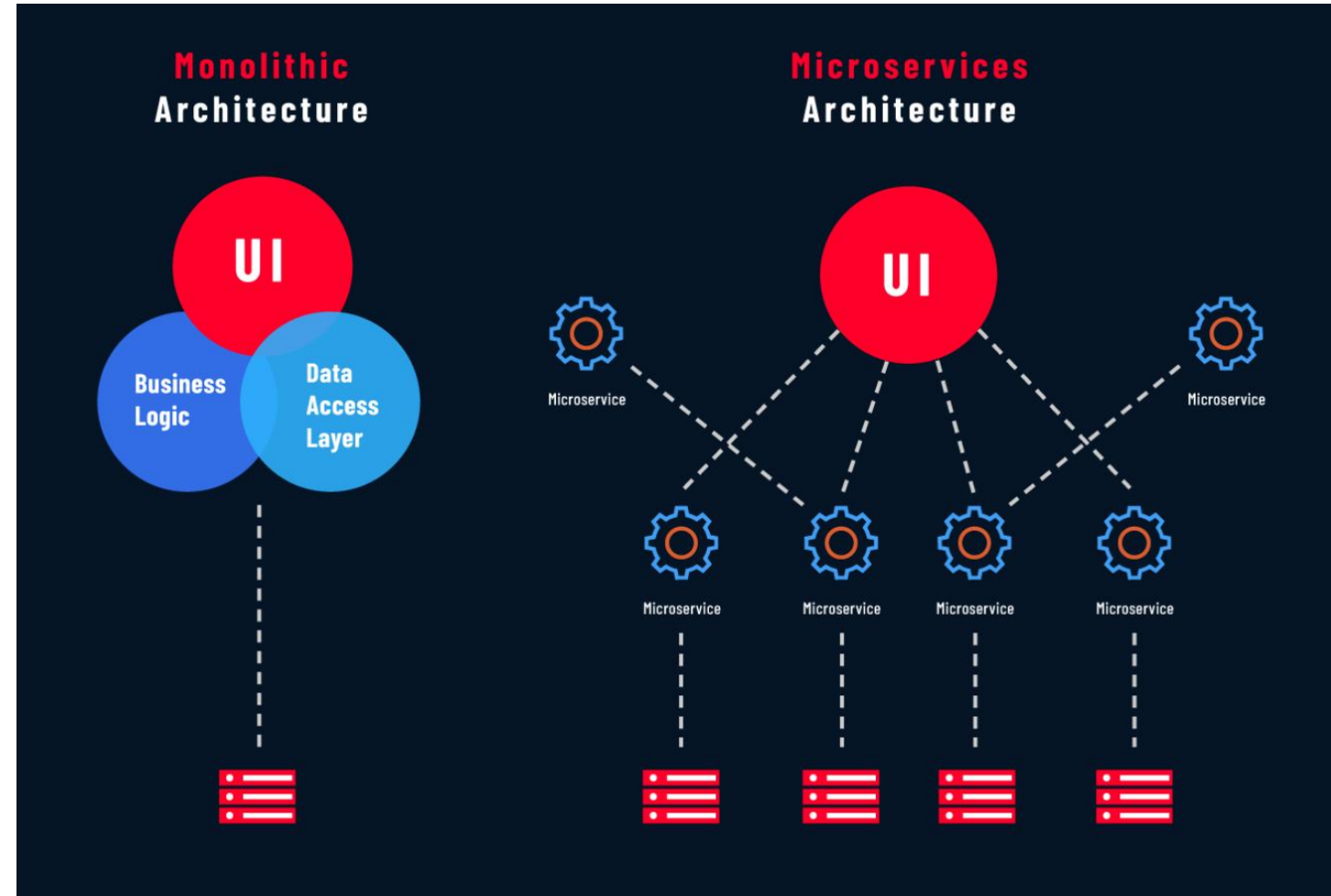


Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Monoliths and Microservices

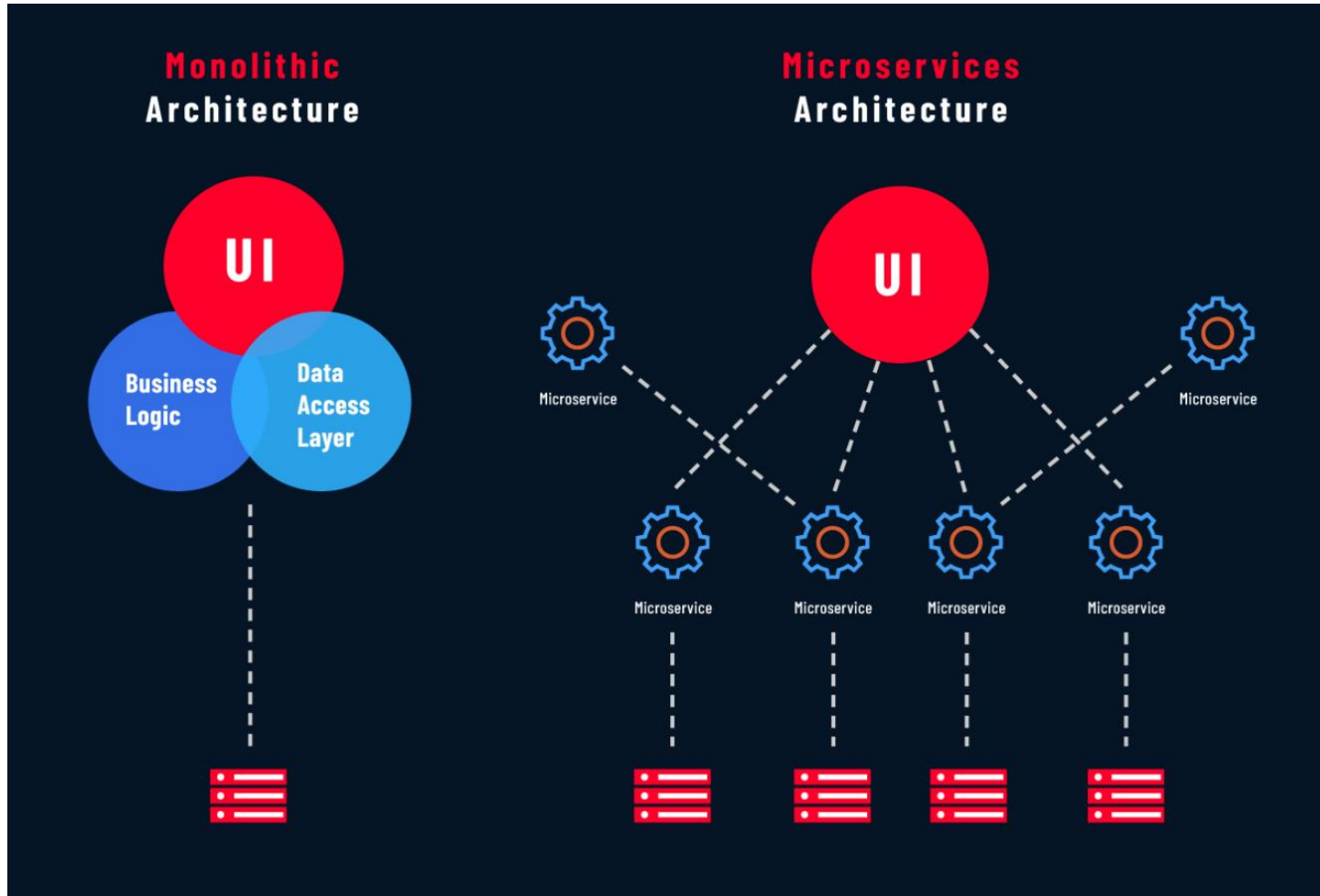
Monolith

- Everything under one codebase.
- Often tightly coupled.
- Hard to modify or upgrade.
- Poor modularity.
- Difficult to reuse components.
- Changes can create unintended consequences.
- Two types of coupling in monoliths:
 - Technical coupling (layer dependencies)
 - Domain coupling (business domains tied together)
- Monoliths may:
 - Use multitier architecture
 - Still tightly couple multiple domains



Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Monoliths and Microservices



Microservice

- Decentralized
- Loosely coupled
- Independent services
- Each service has a single responsibility
- Benefits:
 - Independent deployment
 - Independent scaling
 - Failure isolation
 - Easier maintenance
 - If one service fails → others keep running.

Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

Monoliths and Microservices

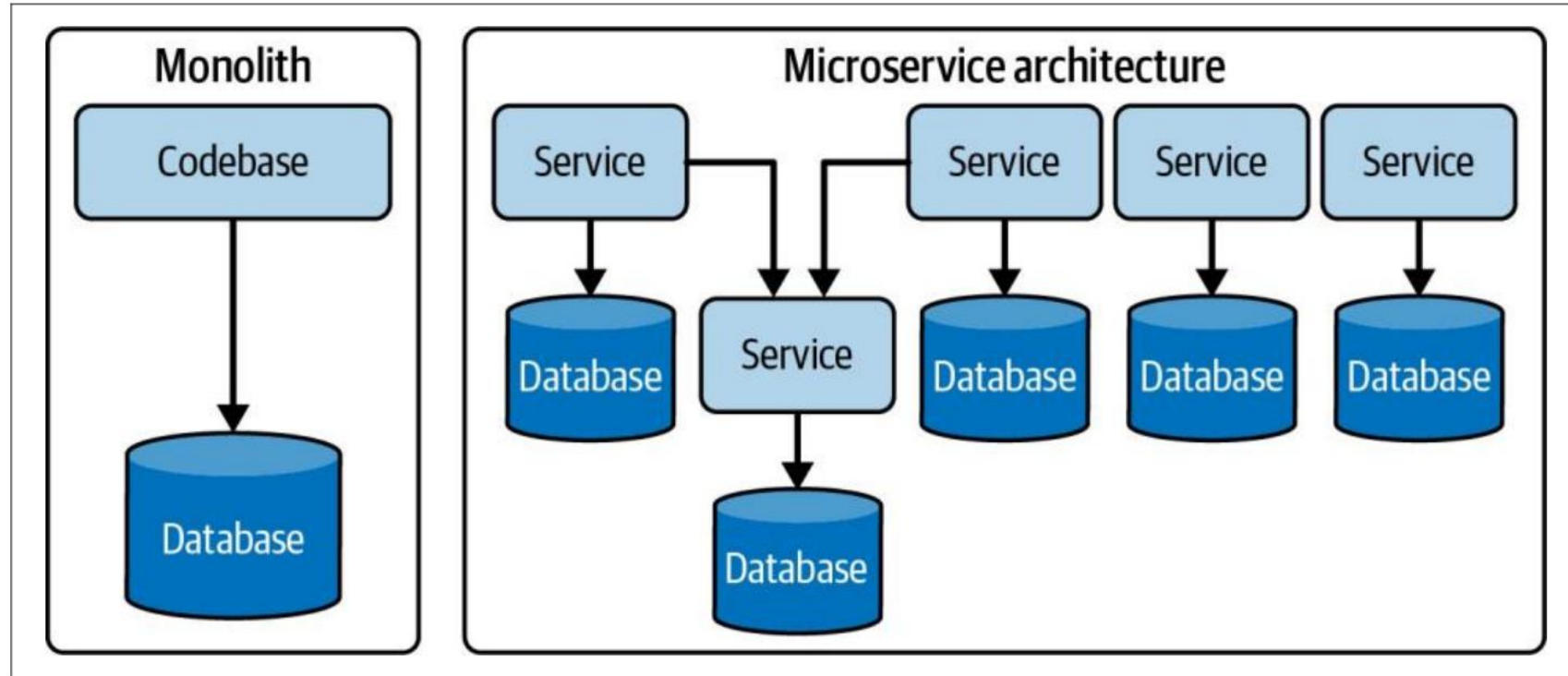


Figure 3-7. An extremely monolithic architecture runs all functionality inside a single codebase, potentially colocating a database on the same host server

User Access: Single vs Multitenant

How to share systems across teams, organizations, or customers.

Multitenancy: Multiple users or groups share the same system while maintaining isolation.

Example: Cloud services, object storage, VMs.

Performance

- Multiple tenants may compete for resources.
- **Noisy neighbor problem:** High usage by one tenant can degrade performance for others.

Security

- External customers should never see each other's data.

Event Driven Architecture

What is an Event?

- An event = something that happens in your business.
- Examples:
 - New customer signup
 - New order placed
 - Order updated
- Often represents a change in state.

EVENT = BUSINESS HAPPENING



REPRESENTS A CHANGE IN STATE

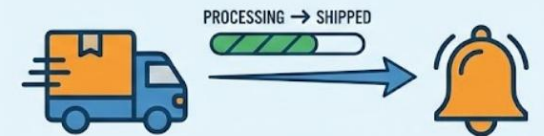
NEW ORDER PLACED



NEW CUSTOMER SIGNUP



ORDER UPDATED



Examples

Event Driven Architecture

Event-Driven Workflow

Three main parts:

- Event Production:
 - Create the event (e.g., order created)
- Event Routing
 - Send the event to interested parties asynchronously
- Event Consumption
 - Services consume the event without tight dependencies on the producer

Main principle:

Loose coupling between producer, router, and consumer.



Figure 3-8. In an event-driven workflow, an event is produced, routed, and then consumed

Event Driven Architecture

Event-Driven Architecture

- Architecture that uses the event-driven workflow across services.
- Distributes state of events across multiple services.
- Advantages:
 - Services can go offline without losing events.
 - Multiple services or consumers can access the same events.
 - Works well with loosely coupled services.

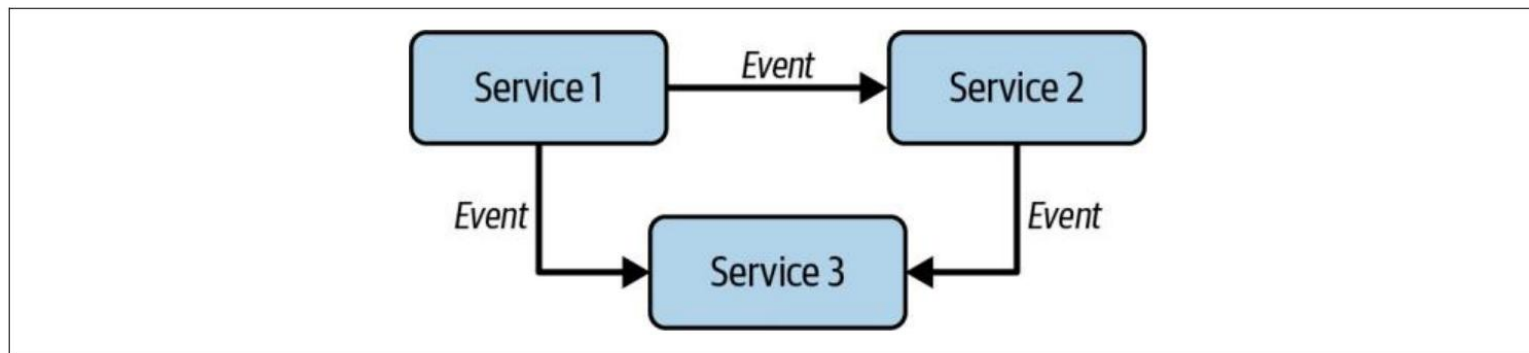


Figure 3-9. In an event-driven architecture, events are passed between loosely coupled services

Brownfield vs Greenfield Projects

Brownfield Projects

- Refactoring or reorganizing legacy architecture
- Constrained by past decisions and technologies

Better Approach

- Gradually replace parts of the old system
- Incremental, low-risk transition
- Allows reversible decisions
- Test impact before full replacement

Greenfield Projects

- Starting from scratch
- No legacy constraints
- Clean slate

Best Practices

- Prioritize business requirements
- Focus on value, not novelty